

tasks-template

description: “Task list template for feature implementation”

Tasks: [FEATURE NAME]

Input: Design documents from /specs/[###-feature-name]/

Prerequisites: plan.md (required), spec.md (required for user stories), research.md, data-model.md, contracts/

Tests: The examples below include test tasks. Tests are OPTIONAL - only include them if explicitly requested in the feature specification.

Organization: Tasks are grouped by user story to enable independent implementation and testing of each story.

Format: [ID] [P?] [Story] Description

- **[P]:** Can run in parallel (different files, no dependencies)
- **[Story]:** Which user story this task belongs to (e.g., US1, US2, US3)
- Include exact file paths in descriptions

Path Conventions

- **Single project:** src/, tests/ at repository root
- **Web app:** backend/src/, frontend/src/
- **Mobile:** api/src/, ios/src/ or android/src/
- Paths shown below assume single project - adjust based on plan.md structure

Phase 1: Setup (Shared Infrastructure)

Purpose: Project initialization and basic structure

☐

T001 Create project structure per implementation plan

☐

T002 Initialize [language] project with [framework] dependencies

☐

T003 [P] Configure linting and formatting tools

Phase 2: Foundational (Blocking Prerequisites)

Purpose: Core infrastructure that **MUST** be complete before ANY user story can be implemented

CRITICAL: No user story work can begin until this phase is complete

Examples of foundational tasks (adjust based on your project):

☐

T004 Setup database schema and migrations framework

☐

T005 [P] Implement authentication/authorization framework

☐

T006 [P] Setup API routing and middleware structure

☐

T007 Create base models/entities that all stories depend on

☐

T008 Configure error handling and logging infrastructure

☐

T009 Setup environment configuration management

Checkpoint: Foundation ready - user story implementation can now begin in parallel

Phase 3: User Story 1 - [Title] (Priority: P1) MVP

Goal: [Brief description of what this story delivers]

▲
Independent Test: [How to verify this story works on its own]

Tests for User Story 1 (OPTIONAL - only if tests requested)

NOTE: Write these tests FIRST, ensure they FAIL before implementation

☐

T010 [P] [US1] Contract test for [endpoint] in tests/contract/test_[name].py

☐

T011 [P] [US1] Integration test for [user journey] in tests/integration/test_[name].py

Implementation for User Story 1

☐

T012 [P] [US1] Create [Entity1] model in src/models/[entity1].py

☐

T013 [P] [US1] Create [Entity2] model in src/models/[entity2].py

☐

T014 [US1] Implement [Service] in src/services/[service].py (depends on T012, T013)

☐

T015 [US1] Implement [endpoint/feature] in src/[location]/[file].py

☐

T016 [US1] Add validation and error handling

☐

T017 [US1] Add logging for user story 1 operations

Checkpoint: At this point, User Story 1 should be fully functional and testable independently

Phase 4: User Story 2 - [Title] (Priority: P2)

Goal: [Brief description of what this story delivers]

Independent Test: [How to verify this story works on its own]

Tests for User Story 2 (OPTIONAL - only if tests requested)

☐

T018 [P] [US2] Contract test for [endpoint] in tests/contract/test_[name].py

☐

T019 [P] [US2] Integration test for [user journey] in tests/integration/test_[name].py

Implementation for User Story 2

☐

T020 [P] [US2] Create [Entity] model in src/models/[entity].py

☐

T021 [US2] Implement [Service] in src/services/[service].py

☐

T022 [US2] Implement [endpoint/feature] in src/[location]/[file].py

☐

T023 [US2] Integrate with User Story 1 components (if needed)

Checkpoint: At this point, User Stories 1 AND 2 should both work independently

Phase 5: User Story 3 - [Title] (Priority: P3)

Goal: [Brief description of what this story delivers]

Independent Test: [How to verify this story works on its own]

Tests for User Story 3 (OPTIONAL - only if tests requested)

☐

T024 [P] [US3] Contract test for [endpoint] in tests/contract/test_[name].py

☐

T025 [P] [US3] Integration test for [user journey] in tests/integration/test_[name].py

Implementation for User Story 3

- ☐ T026 [P] [US3] Create [Entity] model in src/models/[entity].py
- ☐ T027 [US3] Implement [Service] in src/services/[service].py
- ☐ T028 [US3] Implement [endpoint/feature] in src/[location]/[file].py

Checkpoint: All user stories should now be independently functional

[Add more user story phases as needed, following the same pattern]

Phase N: Polish & Cross-Cutting Concerns

Purpose: Improvements that affect multiple user stories

- ☐ TXXX [P] Documentation updates in docs/
 - ☐ TXXX Code cleanup and refactoring
 - ☐ TXXX Performance optimization across all stories
 - ☐ TXXX [P] Additional unit tests (if requested) in tests/unit/
 - ☐ TXXX Security hardening
 - ☐ TXXX Run quickstart.md validation
-

Dependencies & Execution Order

Phase Dependencies

- **Setup (Phase 1):** No dependencies - can start immediately
- **Foundational (Phase 2):** Depends on Setup completion - BLOCKS all user stories
- **User Stories (Phase 3+):** All depend on Foundational phase completion
 - User stories can then proceed in parallel (if staffed)
 - Or sequentially in priority order (P1 → P2 → P3)

- **Polish (Final Phase):** Depends on all desired user stories being complete

User Story Dependencies

- **User Story 1 (P1):** Can start after Foundational (Phase 2) - No dependencies on other stories
- **User Story 2 (P2):** Can start after Foundational (Phase 2) - May integrate with US1 but should be independently testable
- **User Story 3 (P3):** Can start after Foundational (Phase 2) - May integrate with US1/US2 but should be independently testable

Within Each User Story

- Tests (if included) MUST be written and FAIL before implementation
- Models before services
- Services before endpoints
- Core implementation before integration
- Story complete before moving to next priority

Parallel Opportunities

- All Setup tasks marked [P] can run in parallel
- All Foundational tasks marked [P] can run in parallel (within Phase 2)
- Once Foundational phase completes, all user stories can start in parallel (if team capacity allows)
- All tests for a user story marked [P] can run in parallel
- Models within a story marked [P] can run in parallel
- Different user stories can be worked on in parallel by different team members

Parallel Example: User Story 1

Launch all tests for User Story 1 together (if tests requested):

Task: "Contract test for [endpoint] in tests/contract/
test_[name].py"

Task: "Integration test for [user journey] in tests/integration/
test_[name].py"

Launch all models for User Story 1 together:

Task: "Create [Entity1] model in src/models/[entity1].py"

Task: "Create [Entity2] model in src/models/[entity2].py"

Implementation Strategy

MVP First (User Story 1 Only)

1. Complete Phase 1: Setup
2. Complete Phase 2: Foundational (CRITICAL - blocks all stories)
3. Complete Phase 3: User Story 1
4. **STOP and VALIDATE**: Test User Story 1 independently
5. Deploy/demo if ready

Incremental Delivery

1. Complete Setup + Foundational → Foundation ready
2. Add User Story 1 → Test independently → Deploy/Demo (MVP!)
3. Add User Story 2 → Test independently → Deploy/Demo
4. Add User Story 3 → Test independently → Deploy/Demo
5. Each story adds value without breaking previous stories

Parallel Team Strategy

With multiple developers:

1. Team completes Setup + Foundational together
 2. Once Foundational is done:
 - Developer A: User Story 1
 - Developer B: User Story 2
 - Developer C: User Story 3
 3. Stories complete and integrate independently
-

Notes

- [P] tasks = different files, no dependencies
- [Story] label maps task to specific user story for traceability
- Each user story should be independently completable and testable
- Verify tests fail before implementing
- Commit after each task or logical group
- Stop at any checkpoint to validate story independently
- Avoid: vague tasks, same file conflicts, cross-story dependencies that break independence